

2025 Fall CSED451 Assignment3-1 Report

Team Name: openGameLab

Team Member #1: 안재영

컴퓨터공학과/20220019/enter21

Team Member #2: 정윤희

컴퓨터공학과/20240385/jyhyeok

1. Technical Details

개발 환경: Visual Studio 2022 버전 17.14.9 (July 2025)

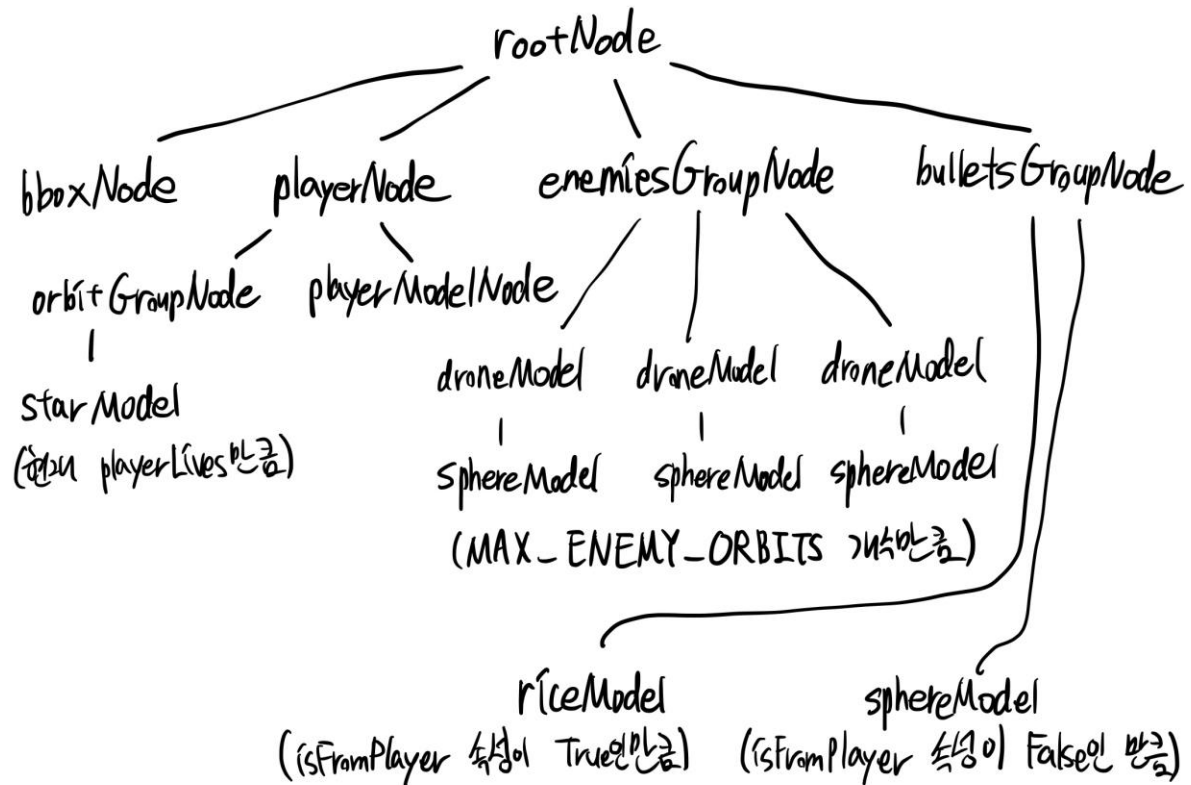
cpp 확장자 파일을 powershell 스크립트 파일로 만든 후 powershell에서 빌드하여 구현 결과를 확인할 수 있다. 또한, OpenGL을 사용하는 코딩에 익숙해야 한다.

2. Implementation Details

2-1. Outline

Assignment3-1은 Assignment1과 Assignment2에서 구현했던 Bullet Hell shooter의 로직들을 그대로 유지하면서, 3D 그래픽 스타일, 카메라 렌더링, 씬 그래프를 추가하는 것이 주 목표이다. 플레이어, 적, bullet 등 텍스트를 제외한 모든 엔티티가 3D 모델로 표현되었으며, 제공된 assets 파일의 3D 모델들을 적극적으로 활용했다. 이를 위해 Model 클래스를 도입하여 3D 모델을 로드하거나 그리는 기능을 구현했으며, 추가적으로 drawPlayer, drawBullets 등 기존 draw 함수들을 3D 모델을 활용하여 그리도록 변경하였다.

단일 씬 그래프로 모든 엔티티를 표현하기 위해 Node 구조체를 구현하였다. Node 구조체에는 위치, 회전, 스케일링, 색깔 정보를 저장하는 벡터들, 그림을 그릴 모델의 포인터, 자식 노드들의 정보, 그리고 기타 기능들을 가지고 있다. 모든 엔티티들은 노드로 저장되어 있다. 이 Node 구조체에서 drawRecursive 함수를 구현하여, rootNode부터 시작하여 모든 노드에 순회를 돌아 현재 화면 상의 모든 엔티티를 그린다. 간략하게 씬 그래프를 그리면 아래 이미지와 같다.



Q키를 눌러 그래픽 스타일을 바꿀 수 있다. 이번 과제에서는 우선 opaque polygon style과 wireframe style 두 가지 그래픽 스타일을 구현한다. 이를 위해 setGraphicStyle이라는 함수를 추가하여 currentGraphicStyle이라는 변수에 따라 그래픽 스타일을 바꾸도록 했으며, handleKeyDown 함수에 관련 키 상호작용을 구현했다.

C키를 눌러 카메라 뷰 타입을 바꿀 수 있다. 총 세 가지 뷰 타입을 구현하였으며, 각각 top view with perspective projection, top view with orthographic projection, third-person view with perspective projection이다. 이를 위해 setCameraViews 함수를 추가하여 currentCameraView라는 변수에 따라 카메라 뷰를 바꾸도록 했으며, handleKeyDown 함수에 관련 키 상호작용을 구현했다.

3D 그래픽 방식과 다양한 카메라 뷰 추가로 인해, 기존에 사용하던 윈도우 창을 제한 구역으로 하기에는 플레이어가 이동 제한 구역을 잘 인지하지 못한다는 문제가 발생한다. 따라서 과제 문서에 따라 플레이어가 움직일 수 있는 공간인 bounding box를 구현하였으며, 노란색 선으로 구역을 잘 보이게 하였다. 이 bounding box 또한 씬 그래프에 의해 노드로 관리되어 그려진다. Bullet들도 이 bounding box 외부로 이동하면 사라진다.

2-2. Additional Requirements

- 적 색과 HP 연동

Enemy 객체의 health와 maxHealth 값을 가져와 현재 남은 체력의 비율을 계산하고, 선형 보간 함수 lerp를 사용해 healthyColor와 damagedColor 사이에서 비율에 맞는 중간 색을 계산한다. 함수에 의해 반환된 색상 값을 적의 color 속성에 덮어씌워서 HP의 변화를 색깔로 인지할 수 있도록 한다.

- 적의 플레이어 타겟팅

Enemy::update 함수에서 매 프레임마다 적의 현재 위치를 기반으로 플레이어와의 x, y 좌표의 차이를 계산한다. 그 후 atan2 함수를 통해 적이 플레이어를 향하는 벡터의 각도를 계산하고, Enemy 구조체 내의 targetAngle에 저장한다. 적을 실제로 그리는 updateSceneGraph 함수에서 enemyNode->rot.y에 해당 각도를 대입하여 drawRecursive가 해당 노드를 그릴 때 플레이어를 향한 방향으로 정렬되게 구현하였다.

- 플레이어 총알 명중 파티클

handleCollision 함수가 플레이어 총알과 적의 충돌을 감지한 순간에 총알이 맞은 위치에서 8개 방향으로 작은 노란색 파티클이 퍼져나가도록 한다. 이를 위해 위치, 속도, 생존 시간과 색깔 인자를 가지는 구조체 Particle을 새롭게 추가한다.

생성 함수 bulletParticleEffect는 파티클을 생성할 피격 위치 x, y를 인자로 받으며, 이를 기준으로 8방향으로 퍼져나가는 원형 파티클들을 생성해 velocity와 lifetime을 할당한다. 이 때 lifetime에는 rand()를 통한 약간의 랜덤성을 부여해 연출의 생동감을 부여한다. 생성된 파티클은 particles에 저장되어, 이후 updateParticles에 의해 관리되고 drawParticleEffect에 의해 그려진다.

updateParticles는 timer에 의해 호출되어 매 프레임마다 particles 속 파티클들의 위치를 속도를 기반으로 갱신하고, 생존 시간을 감소시키다가 수명이 다할 경우 삭제한다. drawParticleEffect는 particles 속 파티클들의 위치와 색을 기반으로 파티클들을 그리는데, 이 때 GL_BLEND를 활성화시켜 컬러 블렌딩을 통해 glBlendFunc(GL_SRC_ALPHA, GL_ONE)의 가산 혼합 설정을 적용한다. 이는 파티클들이 겹칠수록 더 밝아지는 효과를 통해 파티클이 빛나는 듯한 연출을 효과적으로 표현한다. 또한 glDisable(GL_DEPTH_TEST)로 깊이 테스트를 비활성화해 파티클이 다른 3D 모델에 가려지지 않도록 보장한다.

- 플레이어 추진 파티클

플레이어가 W키를 통해 앞으로 추진할 때 기체 뒤쪽에서 파티클이 분출되도록 한다. 이 파티클들은 boostParticleEffect 함수를 통해 생성되며, processInput의 'w' 입력 인식 부분에서 호출된다. 플레이어 제트기 모델의 후방 위치에서 1~2개의 파티클을 생성해 particles에 추가하고, 파티클의 속도는 플레이어의 뒤쪽을 향하도록 마찬가지로 rand()를 통해 약간의 랜덤성을 준다. 파티클의 관리는 위의 총알 명중 효과와 동일한 함수들에 의해 처리된다.

- 적 주위 공전 엔티티

게임의 3D 입체감을 조금 더 살릴 수 있도록, 적의 주위를 공전하는 엔티티를 구현하여 게임이 단순 평면만을 그리고 있는 것이 아님을 강조하였다. 이를 위해 씬 그래프의 관계에서 적의 자식에 새로운 노드를 추가하였고, main 함수에서 이를 위한 enemyOrbitNodePool을 생성하였다.

updateSceneGraph 함수에서 enemyNode를 enemiesGrounNode에 추가하기 전에 enemyOrbitNodePool에서 공전 엔티티 노드를 가져와 children.push_back()으로 enemyNode의 자식으로 추가하고, glutGet(GLUT_ELAPSED_TIME)을 인자로 삼각함수를 호출해 공전 엔티티들의 x, y좌표에 대입하였다. 이를 통해 평면을 뚫고 각 enemyNode의 로컬 XY 평면을 공전하는 4개의 구체를 구현하였다.

2-3. Design Rationale

이번 과제에서는 이후 더 많은 모델 추가 및 유지보수에 대비하여 3D 그래픽을 위한 환경 설정, 다양한 그래픽 스타일 및 카메라 뷰 구현을 위한 토대를 마련하고자 하였다. 기존 display 함수는 각 엔티티를 그리는 함수가 다 따로 있었고, vertex array에 점들을 저장하여 그것을 불러오는 방식으로 그림을 그렸다. 이는 이후 그려야 할 엔티티가 추가되거나 삭제될 때 관리가 불편하게 되고, 엔티티가 굉장히 많아졌을 때 부하를 일으킬 가능성이 있다. 이에 아래와 같은 방식으로 코드를 추가하고 개선하였다.

1) 3D 모델 환경 도입: 3D 모델을 관리하는 Model 클래스를 구현하여, 3D 모델 로드 기능과 draw 기능을 구현했다. 외부 라이브러리인 tiny_obj_loader.h 헤더 파일을 include하여, 복잡한 구현을 피하고자 하였다. 또한 기존 2D display 방식에 depth 속성들을 추가하여 3D로 그림을 그릴 수 있도록 환경을 세팅했다.

2) 씬 그래프: Node 구조체를 구현하여, 엔티티들의 성질들을 벡터로 저장하고, tree 형태로 노드들을 관리한다. 텍스트를 제외한 모든 엔티티들은 노드로 관리된다. 이에 따라 이후 엔티티 추가나 삭제에 편리하고 기민하게 대응할 수 있게 되었다. display함수 또한 rootNode만 그리는 것으로 변경하고, rootNode부터 시작하여 tree의 모든 노드들을 drawRecursive라는 함수를 통해 순회하며 그리도록 하였다.

2-4. Design Implementation

Model 클래스는 에셋 파일에 저장된 obj 파일들을 읽어와서 파일에 저장된 점 정보들을 저장하는 load 함수, 그리고 모델을 실제로 그리는 draw 함수로 구성된다.

Node 구조체는 pos, rot, scale, color 네 가지 Vec3 구조체 벡터들을 가지며 이것들은 각 노드의 위치, 회전, 스케일, 색깔을 나타내어 노드별로 정보를 저장하도록 한다. 어떤 모델로 그려질지와, 자식 노드들의 정보, *customDrawFunc이라는 함수 포인터, isVisible이라는 bool타입 변수를 가지고 있다. *customDrawFunc은 draw 함수를 노드와 연결하는 기능을 하고, isVisible bool타입 변수는 리스폰 대기시간이나 적 처치 시 더 이상 그리지 않게 하기 위해 존재한다.

Node 구조체들을 원소로 가지는 벡터인 Node Pool들이 존재한다. 총 네 개가 존재하며, 각각 orbitEntityNodePool, enemyNodePool, bulletNodePool, enemyOrbitEntityNodePool이 존재한다. 순서대로 플레이어 주변을 회전하는 엔티티, 적 엔티티, bullet 엔티티, 적 주변을 회전하는 엔티티들이 저장된다. bulletNodePool이 Node Pool을 만든 주요 원인인데, Node 개수가 딱 정해지거나 5개 이하인 다른 Node Pool들과는 다르게 bulletNodePool은 bullet이 매우 많을 수 있기 때문에, 이를 각각 하나씩의 노드로 직접 관리하는 데에는 어려움이 있었기 때문이다.

기존 draw함수들은 vertex array에 저장된 점들의 값을 불러와 그림을 그렸으나, 이제는 적당히 알맞은 3D 모델을 로드하여 그림을 그리도록 바꾸었다. 그 결과 각 draw 함수의 코드가 굉장히 짧아졌다.

drawBoundingBox 함수는 이번 과제에서 새로 추가한 함수이다. 이동 제한 구역 크기는 이전 과제들과 같다. 정육면체 크기이며, 카메라 뷰에 따라 bounding box가 다르게 보이지만, 실제 크기는 항상 동일하다.

setGraphicStyle과 setCameraViews 함수 또한 이번 과제에서 요구하는 minimum goal을 충족하기 위해 구현한 함수이다. 각각 currentGraphicStyle과 currentCameraView 변수를 사용하며, Q키와 C키를 누를 때마다 변수값이 바뀐다. 이 유동적인 변수값을 setGraphicStyle과 setCameraViews 함수들에서 이용하여 플레이어 입력에 따라 그래픽

스타일과 카메라 뷰를 적절히 바꾼다. 현재 구현한 그래픽 스타일은 opaque polygon style과 wireframe style이며, 구현한 카메라 뷰는 top view with perspective projection, top view with orthographic projection, third-person view with perspective projection이다. 현재 그래픽 스타일과 카메라 뷰를 인지하기 쉽도록, 좌상단에 텍스트로 현재 그래픽 스타일과 카메라 뷰를 확인할 수 있도록 하였다.

setGraphicStyle 함수에서는 glPolygonMode를 이용한다. setCameraViews 함수에서는 카메라 뷰에 따라 gluPerspective, glOrtho, gluLookAt를 적절히 이용한다.

updateSceneGraph는 현재 게임 상황에 맞게 씬 그래프를 업데이트하는 함수이다. 플레이어 현재 위치, 플레이어 주변 회전 엔티티, 적 엔티티, bullet 엔티티들이 여기서 관리된다. 특히 Node Pool로 관리하는 엔티티들은, 자식 노드들을 Clear한 뒤 Node Pool에 저장된 정보들로 다시 노드를 만들어 업데이트하는 방식을 취했다.

display함수는 씬 그래프 방식을 도입했기에, rootNode.drawRecursive()와 2D 텍스트만을 그리도록 변경하여 획기적으로 코드가 짧아졌고 이해하기 쉬워졌다.

handleKeyDown 함수에서는 새롭게 추가된 그래픽 스타일 및 카메라 뷰를 플레이어가 직접 변경할 수 있도록 Q키와 C키 입력에 따른 코드를 추가했다. 각 키를 누르면, currentGraphicStyle 변수와 currentCameraView 변수가 변화한다.

main 함수에서는 이제 3D 그래픽을 사용하므로 glutInitDisplayMode의 파라미터로 GLUT_DEPTH를 추가하여 3D 뎀스를 주었다. 그리고 glewInit 뒤에 3D 모델 에셋들을 로드하고 Node Pool을 초기화한다. 그 후, push_back을 이용하여 씬 그래프 tree를 만들어 준다.

2-5. Additional Background

본 구현을 위해서는 OpenGL에서 3D 공간을 표현하는 방법, 그리고 씬 그래프 자료 구조에 대한 전반적인 이해가 필요했다.

1) 3D 카메라 변환: 3D 공간을 2D 공간으로 표현하기 위해 두 가지 핵심적인 행렬 변환을 활용하였다. 첫번째는 카메라의 위치와 시점 방향을 설정하는 시점 변환 gluLookAt, 두번째는 3D 공간에 원근감을 부여하는 투영 변환 gluPerspective와 glOrtho였다. 이들을 구분해 적절하게 사용하면서 3D 공간을 바라보는 3가지 시점을 구현할 수 있었다.

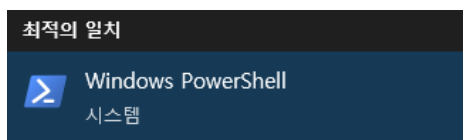
2) 계층적 씬 그래프: 엔티티들을 Node 그래프로 관리하기 위해 씬 그래프 자료구조를 활용하였다. 이 트리를 drawRecursive 함수가 glPushMatrix와 glPopMatrix를 사용해 행렬 상태를 관리하면서 재귀적으로 순회하며, 따라서 플레이어/적의 로컬 좌표계에서 공전하

는 엔티티 등 계층적 움직임을 쉽게 구현할 수 있었다.

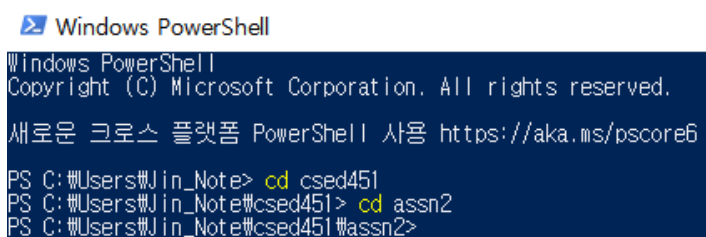
3) 알파 블렌딩과 깊이 버퍼 제어: 파티클 이펙트의 퀄리티를 향상시키기 위해 OpenGL의 상태, 특히 알파 블렌딩과 깊이 버퍼를 추가로 제어하였다. 알파 블렌딩의 가산 혼합은 파티클이 겹칠수록 밝아지도록 하여 빛의 느낌을 강화하였으며, 깊이 테스트의 비활성화는 이펙트가 가려지는 것을 방지할 수 있었다.

3. End-User Guide

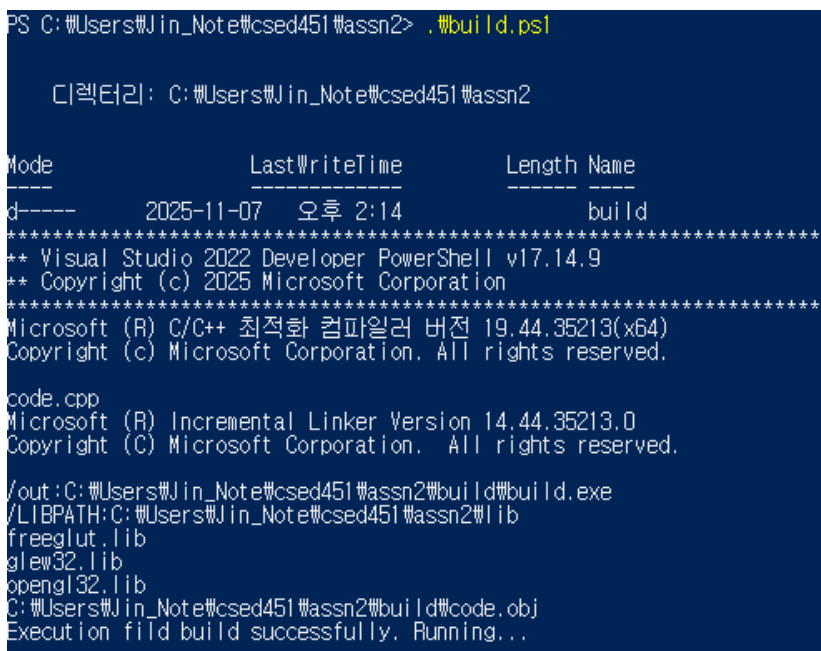
Windows powershell을 실행한다.



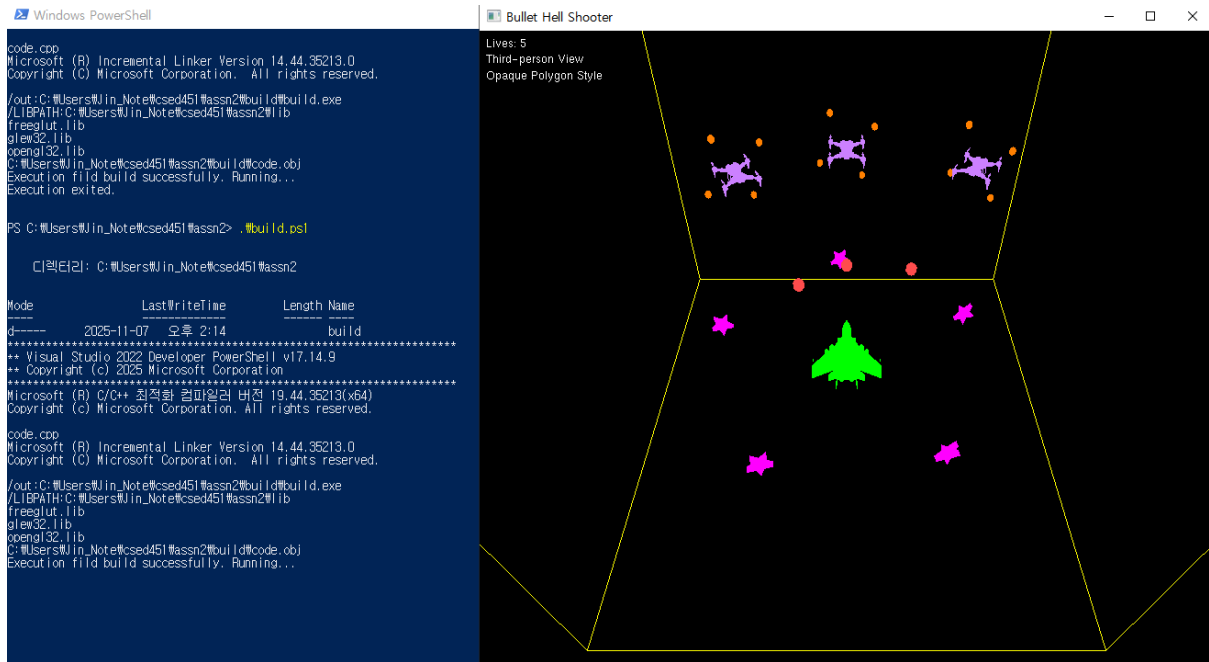
프로젝트가 저장된 파일 위치로 이동한다.



.\build.ps1을 입력한다.



잘 실행되는 것을 확인할 수 있다.



실행된 게임에서는 WASD 키를 통해 플레이어 이동이 가능하며, 스페이스바를 눌러 총알을 발사할 수 있다. Q키를 누르면 그래픽 스타일을, C키를 누르면 카메라 뷰 스타일을 변경할 수 있다.

프로그램 종료 시 powershell 창에는 아래 메시지가 출력된다.

```
Execution exited.

PS C:\Users\Jin_Note\csed451\assn2>
```

이후 다시 빌드하여 프로그램을 실행시키는 것 또한 가능하다.

만약 Powershell 스크립트 실행 정책이 제한되어 있는 등의 이유로 Powershell에서 스크립트를 실행할 수 없다면, Get-ExecutionPolicy 명령어를 입력하여 실행 정책이 제한되어 있는지 확인해야 한다. 만약 그렇다면, Set-ExecutionPolicy RemoteSigned -Scope CurrentUser 명령어를 입력하여 스크립트 실행 정책을 변경해야 한다. 정책 여부 변경을 묻는 메시지가 나타나면 Y를 입력하여 스크립트 실행 정책을 변경하면 실행이 잘 된다.

```
PS C:\Users\Jin_Note> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

실행 규칙 변경
실행 정책은 신뢰하지 않는 스크립트로부터 사용자를 보호합니다. 실행 정책을 변경하면 about_Execution_Policies 도움말
항목 (https://go.microsoft.com/fwlink/?LinkID=135170)에 설명된 보안 위험에 노출될 수 있습니다. 실행 정책을
변경하시겠습니까?
[Y] 예(Y) [A] 모두 예(A) [N] 아니요(N) [L] 모두 아니요(L) [S] 일시 중단(S) [?] 도움말 (기본값은 "N"): Y
PS C:\Users\Jin_Note>
```


4. Discussions/Conclusions

4-1. Obstacles

2D 그래픽에서 3D 그래픽으로 변환하니, 환경 세팅을 처음부터 해줘야 하는 것과, draw 함수들을 모두 알아놔야 했다. 또한 gluPerspective, glOrtho, gluLookAt을 이용하는 3D 카메라 세팅 및 카메라 포지션을 어떻게 해야 적절할지에 대해 고민이 많았다. 이 때문에 적절한 카메라 각도와 위치를 찾는 데 여러 번의 시행착오가 필요했다.

4-2. Improvement Idea

Assignment 문서에서 제시되었던 Destructible enemy parts를 추가적으로 구현하면 게임 플레이어가 조금 더 슈팅 게임이라는 게임 장르에 몰입할 수 있을 것이다. 또한 현재의 움직임은 굉장히 단순한데, 공격 패턴이나 이동 패턴 등을 더 다양하게 설계한다면 게임성을 높일 수 있다.

4-3. Lessons

이번 과제를 통해 2D 그래픽과 3D 그래픽 구현의 난이도 차이가 매우 큰 것을 느낄 수 있었다. 엔티티 좌표뿐만 아니라 카메라 좌표까지 고려해야 했는데, 이 두 개를 동시에 관리하는 것이 굉장히 어려웠으나 그만큼 3D 좌표와 그래픽 구현에 대한 통찰을 얻을 수 있었다.

또한 씬 그래프를 사용하여 노드 단위로 엔티티들을 관리하면서, 확장성과 유지보수의 장점을 다시 한번 깨달을 수 있었다.

5. References

Learn OpenGL - <https://learnopengl.com/>

6. AI-assisted Coding References

이번 과제의 약 60%가 AI(Gemini)의 도움을 받아 구현되었다. 이전과 다르게 씬 그래프를 사용해 게임 구조를 설계하여야 했기 때문에 씬 그래프 업데이트 함수의 초안과 3D 모델 로드 방법을 Gemini의 도움을 받아 구현하였고, 파티클 효과의 적용 과정에서도 Gemini 사용을 통해 더 높은 퀄리티의 연출을 구현할 수 있었다.

7. individual contribution

이번 프로젝트는 두 팀원이 50:50 비율로 각자의 역할을 나누어 충실히 수행하였다. 각 게임 차원을 2D에서 3D로 변경하는 역할과 그렇게 만들어진 기반에 추가적인 기능을 구현하는 역할로 파트를 나누고, 각각의 작업 내용에 대한 보고서를 작성하여 프로젝트를 성공적으로 완수하였다.